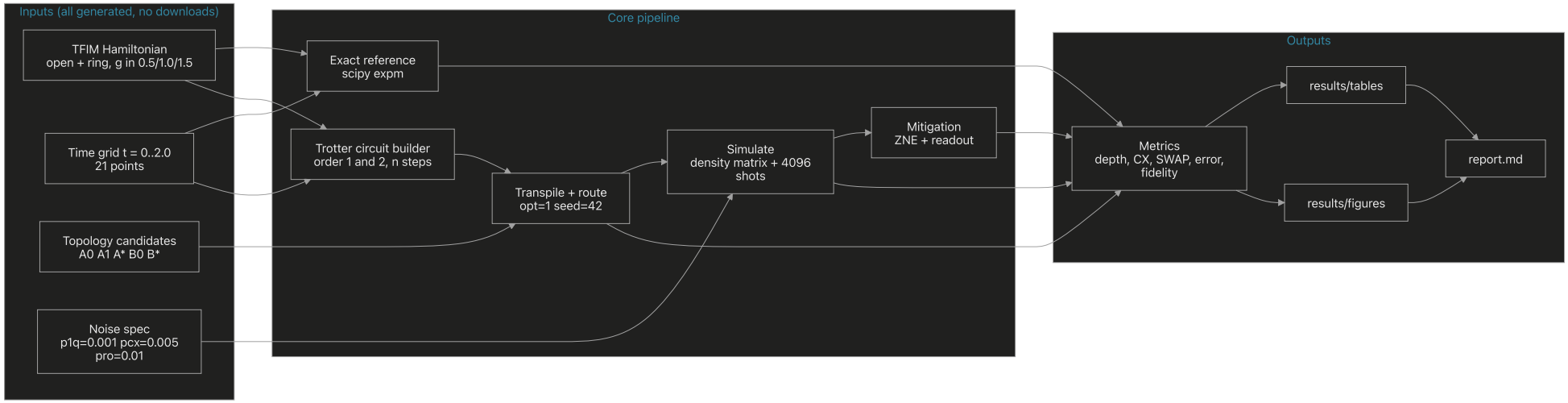
ARCHITECTURE.md – PS-S3 Trotter Noise Co-Design

Companion to `README.md`. The README says **what** and **why**. This file says **how it fits together** and **in what order to build it**. If the two files ever disagree on a number, the README wins.

---

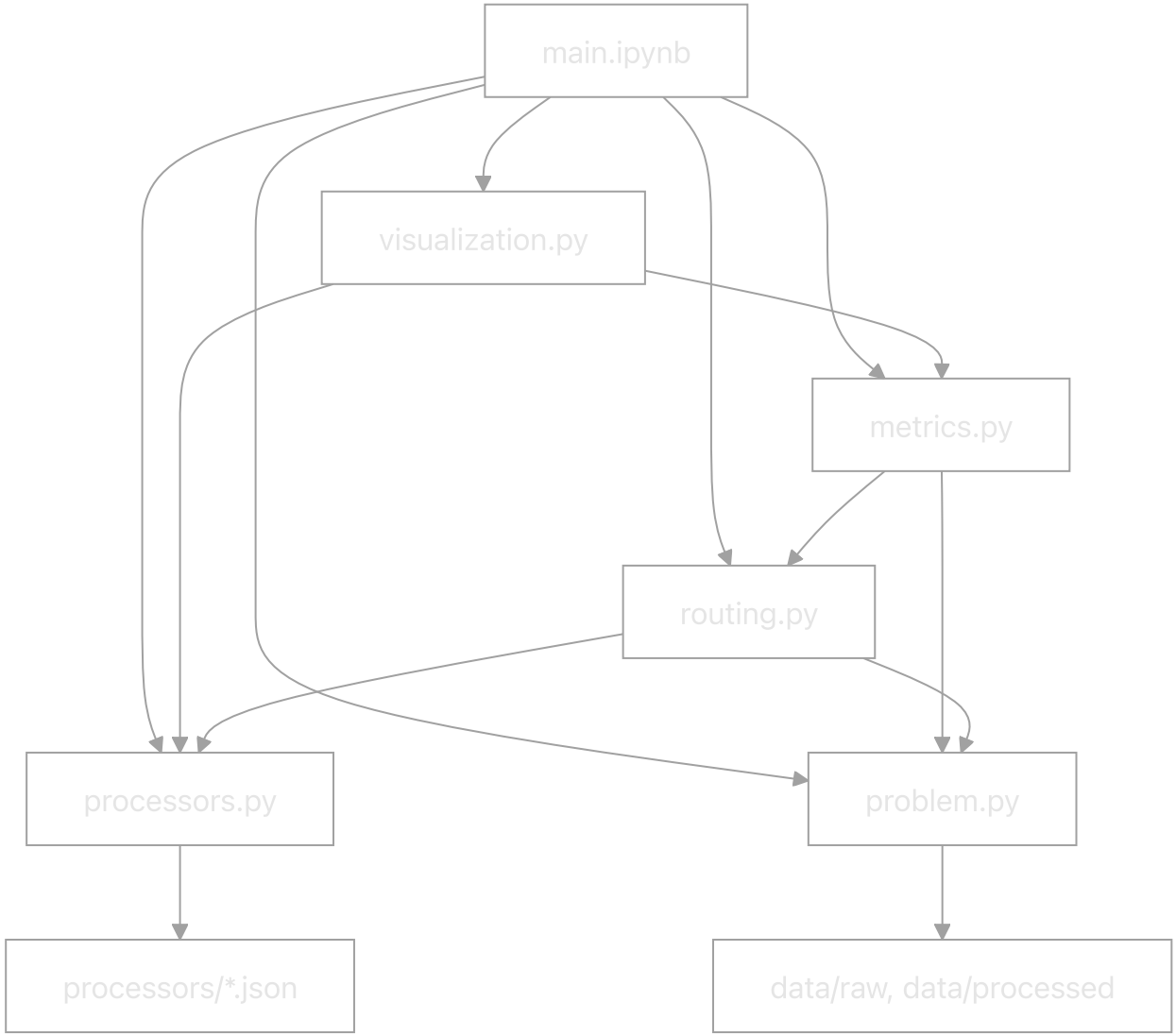
1. System Overview



---

2. Module Dependency Graph

Arrows mean "imports from". There must be **no cycles**.



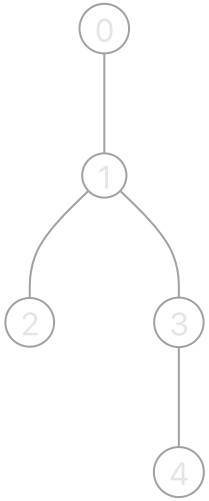
Build bottom-up: `problem` → `processors` → `routing` → `metrics` → `visualization` → notebook.

3. Processor Topologies

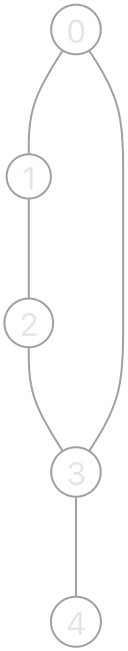
A0 (reference line)



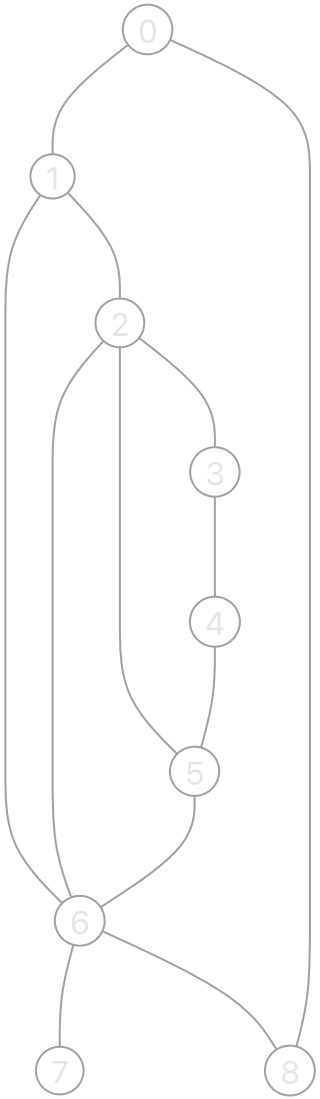
A1 (reference tree)



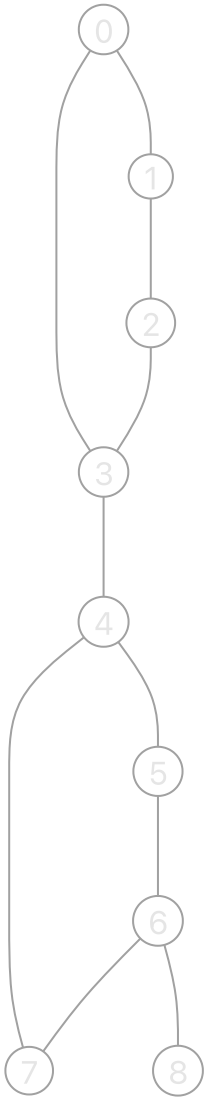
A* (designed: 4-cycle + tail)



B0 (reference, 12 edges)

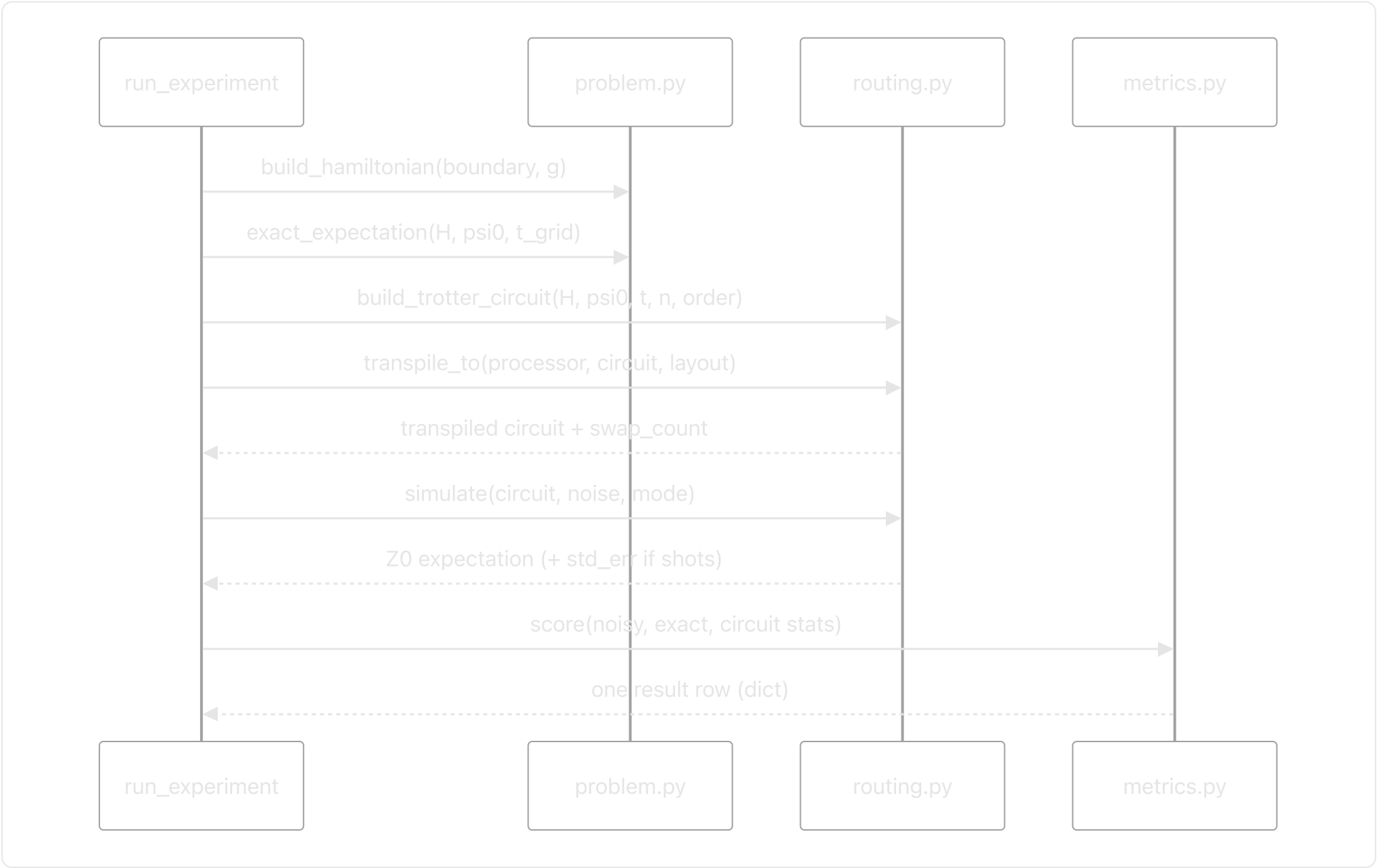


B* (designed: two 4-cycles + bridge + pendant, 10 edges)



Native 4-cycles: A* has 1, B0 has 1 (0-1-6-8), B* has 2. A0 and A1 have none, so the ring TFIM needs SWAPs there.

4. Experiment Flow (one cell of the experiment matrix)



Experiment matrix

Axis	Values
Processor	A0, A1, A*, B0, B*
Boundary	open (required), ring (E1)
g	0.5, 1.0, 1.5
Initial state	`|0000⟩`, Néel `|0101⟩`
Trotter order	1 (required), 2 (E3)
n	1, 2, 4 (required); 1, 2, 3, 4, 6, 8 (E2)
t	21 points, 0 to 2.0
Simulation	density matrix (exact noisy), 4096 shots (spec)

Use a **required subset** (open, order 1, n ∈ {1,2,4}, A0 / B0 and A* / B*) as a first complete run, then widen.

---

5. Module Contracts

All functions are pure where possible, typed, and have docstrings. Row dictionaries use the keys listed in README section 5.

`problem.py`

```python
```

```python
def tfim_hamiltonian(boundary: str, g: float, J: float = 1.0, n: int = 4) -> SparsePauliOp
def initial_state(name: str) -> Statevector          # "zeros" | "neel"
def exact_expectations(H, psi0, t_grid, observables) -> dict[str, np.ndarray]
def save_reference(path: str, ...) -> None           # writes data/processed/*.npy
```

processors.py

python

```python
@dataclass
class Processor: name: str; n_qubits: int; edges: list[tuple[int,int]]; ...
def get_processor(name: str) -> Processor
def graph_metrics(p: Processor) -> dict              # edges, max_degree, diameter, four_cycles, ...
def coupling_map(p: Processor) -> CouplingMap        # bidirectional
def build_noise_model(rz_noisy: bool = False) -> NoiseModel
def write_processor_json(p: Processor, path: str) -> None
```

routing.py

python

```python
def build_trotter_circuit(H, psi0_name, t, n, order) -> QuantumCircuit
def transpile_to(p: Processor, qc, layout=None, seed=42) -> tuple[QuantumCircuit, dict]
    # dict: depth, depth_2q, cx_count, swap_count, total_gates
def count_swaps(p: Processor, qc, layout=None, seed=42) -> int
def layout_search(p: Processor, qc, max_layouts=None) -> pd.DataFrame
def simulate_density(qc_transpiled, p, noise) -> float   # <Z0> incl. analytic readout attenuation
def simulate_shots(qc_transpiled, p, noise, shots=4096) -> tuple[float, float]  # mean, std_err
def zne(qc_transpiled, p, noise, scales=(1,3,5)) -> float
def readout_mitigate(counts, p) -> dict
```

`metrics.py`

```python
def score(noisy, exact) -> dict                          # abs_error, rmse
def state_fidelity_from_dm(rho, psi_exact) -> float
def fit_trotter_constant(errors_noiseless, n_list, t) -> float   # estimates "a"
def predict_n_star(a, t, n_cx, n_1q, p_cx, p_1q) -> float
def run_validation() -> dict[str, bool]                  # the README section 9 tests
```

`visualization.py`

```python
def plot_coupling_map(p: Processor, highlight_layout=None, ax=None)
def plot_trajectories(...)   def plot_u_curve(...)   def plot_gate_scaling(...)
def plot_swap_overhead(...)  def plot_layout_boxes(...)   def plot_mitigation(...)
def plot_summary_nstar(...)
```

Every plot function saves to `results/figures/` at 200 dpi and returns the figure.

---

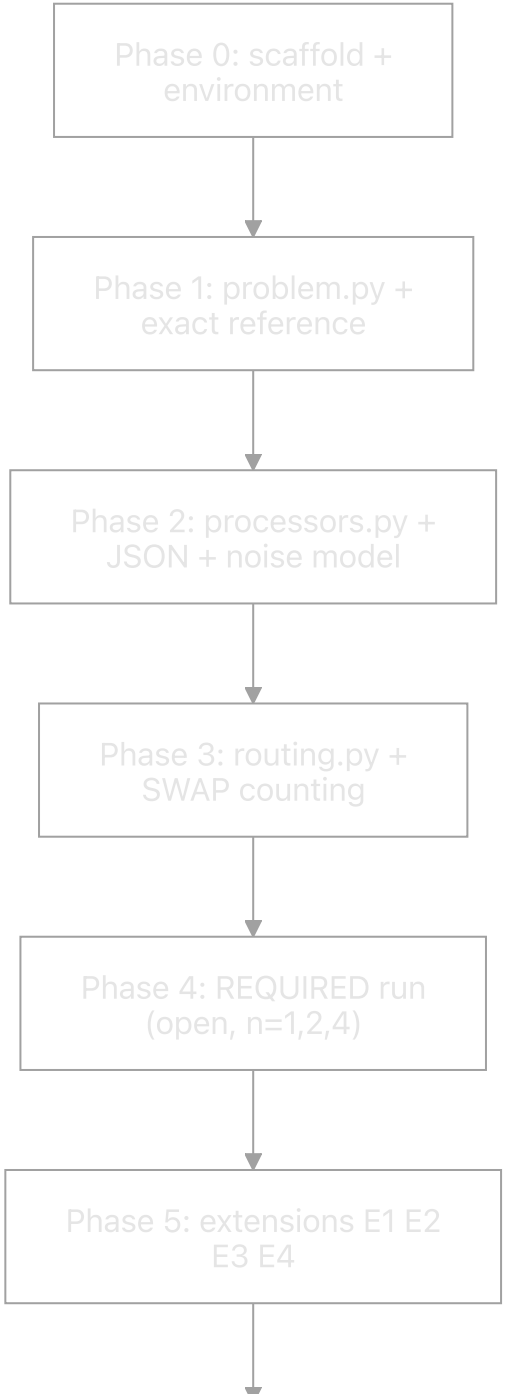
6. Key Implementation Decisions (do not improvise)

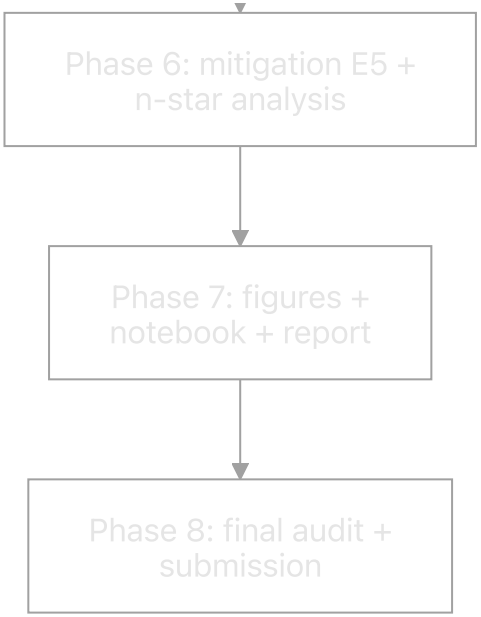
1. **Exact noisy values:** `AerSimulator(method="density_matrix")` with `save_density_matrix`. Readout error is not applied to a saved density matrix, so apply it analytically: each measured qubit in a Z-string of weight `k` attenuates the expectation by `(1 − 2·p_ro)^k`. For $\langle Z_0 \rangle$, `k = 1`.
2. **Shot-based path:** measure only the needed qubit(s), noise model includes `ReadoutError`, 4096 shots, `std_err = sqrt((1 − ⟨Z⟩²)/shots)`.
3. **Noise attaches to physical gates:** add errors to `sx`, `x`, `cx` (and `rz` only in the sensitivity run).

4. **SWAP counting:** transpile with `basis_gates=["rz","sx","x","cx","swap"]` and count `swap`; then transpile again without `swap` for the final depth and CX numbers. Check `cx_final ≈ cx_logical + 3·swap`.
5. **Layout search:** pass `initial_layout` and `optimization_level=1`, `seed_transpiler=42`. Transpile only (no simulation) for all layouts; simulate only the best, median and worst.
6. **Qiskit API:** use `generate_preset_pass_manager` or `transpile`, `PauliEvolutionGate`, `qiskit.synthesis.LieTrotter / SuzukiTrotter`, `qiskit_aer`. Do not use `qiskit.execute`, `BackendV1`, or removed modules.
7. **ZNE:** own implementation by global unitary folding (`U → U (U† U)^k`) with Richardson extrapolation. Mitiq is optional; do not make it a hard dependency.

---

7. Build Order and Phase Gates


Phase 0: scaffold + environment
Phase 1: problem.py + exact reference
Phase 2: processors.py + JSON + noise model
Phase 3: routing.py + SWAP counting
Phase 4: REQUIRED run (open, n=1,2,4)
Phase 5: extensions E1 E2 E3 E4


Phase 6: mitigation E5 + n-star analysis
Phase 7: figures + notebook + report
Phase 8: final audit + submission

Phase	Gate (must pass before moving on)
0	`pip install -r requirements.txt` works in a fresh venv; import check passes
1	H matches a hand-built 2-qubit reference; exact $\langle Z_0(0) \rangle = +1$ for `|0000⟩`; norm preserved
2	5 topologies connected; graph metrics match README; JSON files load back identically
3	Open chain gives SWAP = 0 on A0, A*, B0, B*; ring on A0 gives SWAP > 0; CX identity holds
4	Zero-noise run reproduces noiseless Trotter; required table written; noisy error > noiseless error
5	Ring results differ between topologies; layout study table written
6	`n_star.csv` has predicted and measured columns; mitigation reduces error or the report explains why not
7	All 8 figures exist and are legible; `main.ipynb` runs top to bottom
8	Section 9 of the README: all tests pass; checklist complete

8. Failure Modes and Fixes

Symptom	Likely cause	Fix
A and B give identical numbers on the open chain	Expected (H1). Not a bug	Show the ring extension
Aer rejects the noise model	Error attached to a gate not in the basis	Attach only to `sx, x, cx`
`ImportError` for `execute` / `BackendV1`	Old API in generated code	Use `transpile` + `backend.run`
Density matrix run is very slow	9 qubits × many circuits	Restrict to chosen layouts; reuse transpiled circuits
Different results each run	Missing seeds	Fix `seed_transpiler`, `seed_simulator`, NumPy seed
Shot results disagree with density matrix	Shot noise	Compare within 3 × std_err
ZNE makes the result worse	Folding adds too much noise at small scales	Report honestly; try scales (1, 2, 3)
Notebook takes too long	Full matrix every run	Cache results to `data/processed/`; guard with a `QUICK` flag